

SET 3 A2

- **ID отправки:** 348587032
- **Публичный репозиторий:** <https://github.com/mvdolbilkin/set-3-a2>

1 ArrayGenerator

```
1 class ArrayGenerator {
2 public:
3     static std::vector<int> generateRandomArray(int size, int minVal, int
maxVal) {
4         std::vector<int> arr(size);
5         std::random_device rd;
6         std::mt19937 gen(rd());
7         std::uniform_int_distribution<> distrib(minVal, maxVal);
8         for (int i = 0; i < size; ++i) {
9             arr[i] = distrib(gen);
10        }
11        return arr;
12    }
13
14    static std::vector<int> generateReversedArray(int size) {
15        std::vector<int> arr(size);
16        std::iota(arr.begin(), arr.end(), 1);
17        std::reverse(arr.begin(), arr.end());
18        return arr;
19    }
20
21    static std::vector<int> generateAlmostSortedArray(int size, int swaps) {
22        std::vector<int> arr(size);
23        std::iota(arr.begin(), arr.end(), 1);
24        std::random_device rd;
25        std::mt19937 gen(rd());
26        std::uniform_int_distribution<> distrib(0, size - 1);
27        for (int i = 0; i < swaps; ++i) {
28            std::swap(arr[distrib(gen)], arr[distrib(gen)]);
29        }
30        return arr;
31    }
32};
```

Листинг 1: Класс ArrayGenerator для генерации тестовых данных.

2 SortGenerator

```
1 class SortTester {
2 private:
3     template<typename Func>
4     static long long timeSort(Func sortFunc, std::vector<int>& arr) {
5         auto start = std::chrono::high_resolution_clock::now();
6         sortFunc(arr, 0, arr.size() - 1);
7         auto elapsed = std::chrono::high_resolution_clock::now() - start;
8         return std::chrono::duration_cast<std::chrono::microseconds>(elapsed
).count();
9     }
10
11 public:
12     static double testStandardMergeSort(const std::vector<int>& sourceArr,
int runs) {
13         long long total_time = 0;
14         for (int i = 0; i < runs; ++i) {
15             std::vector<int> arr_copy = sourceArr;
16             total_time += timeSort([&](std::vector<int>& a, int l, int r){
standardMergeSort(a, l, r); }, arr_copy);
17         }
18         return total_time / runs;
19     }
20 }
```

```

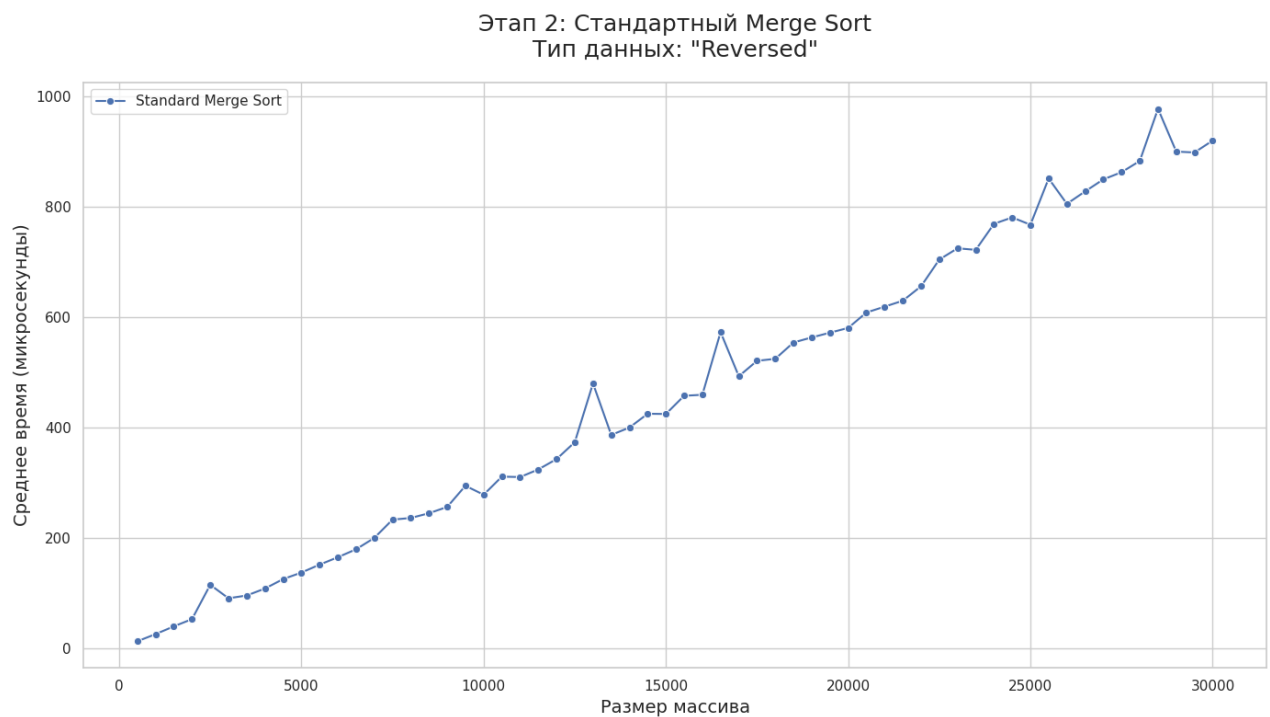
17     }
18     return static_cast<double>(total_time) / runs;
19 }
20
21 static double testHybridMergeSort(const std::vector<int>& sourceArr, int
runs, int threshold) {
22     long long total_time = 0;
23     for (int i = 0; i < runs; ++i) {
24         std::vector<int> arr_copy = sourceArr;
25         total_time += timeSort([threshold](std::vector<int>& a, int l,
int r){ hybridMergeSort(a, l, r, threshold); }, arr_copy);
26     }
27     return static_cast<double>(total_time) / runs;
28 }
29 };

```

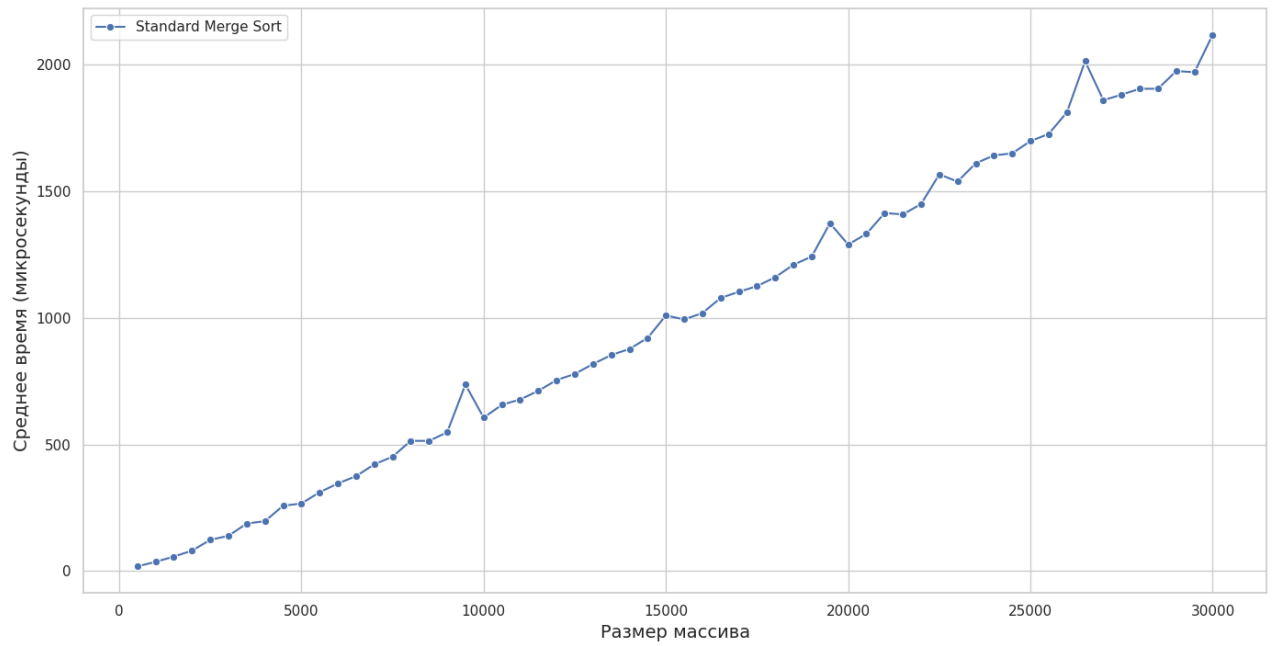
Листинг 2: Класс SortTester для проведения замеров.

3 Графики

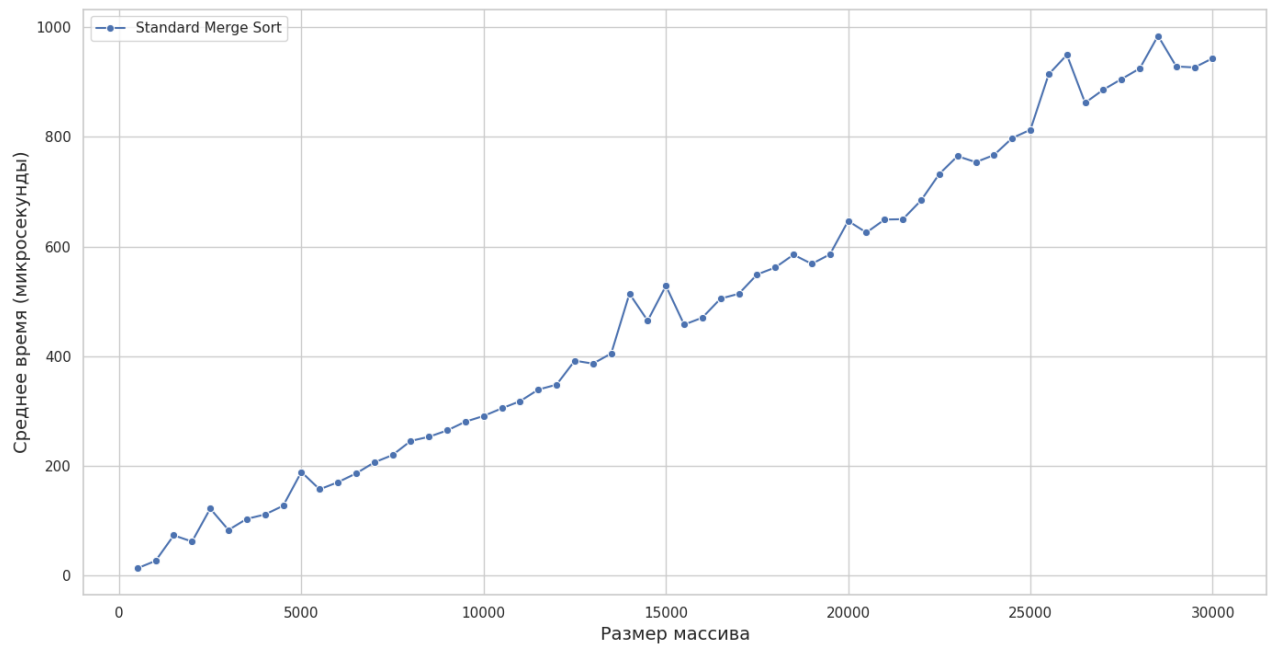
3.1 Для merge sort



Этап 2: Стандартный Merge Sort
Тип данных: "Random"

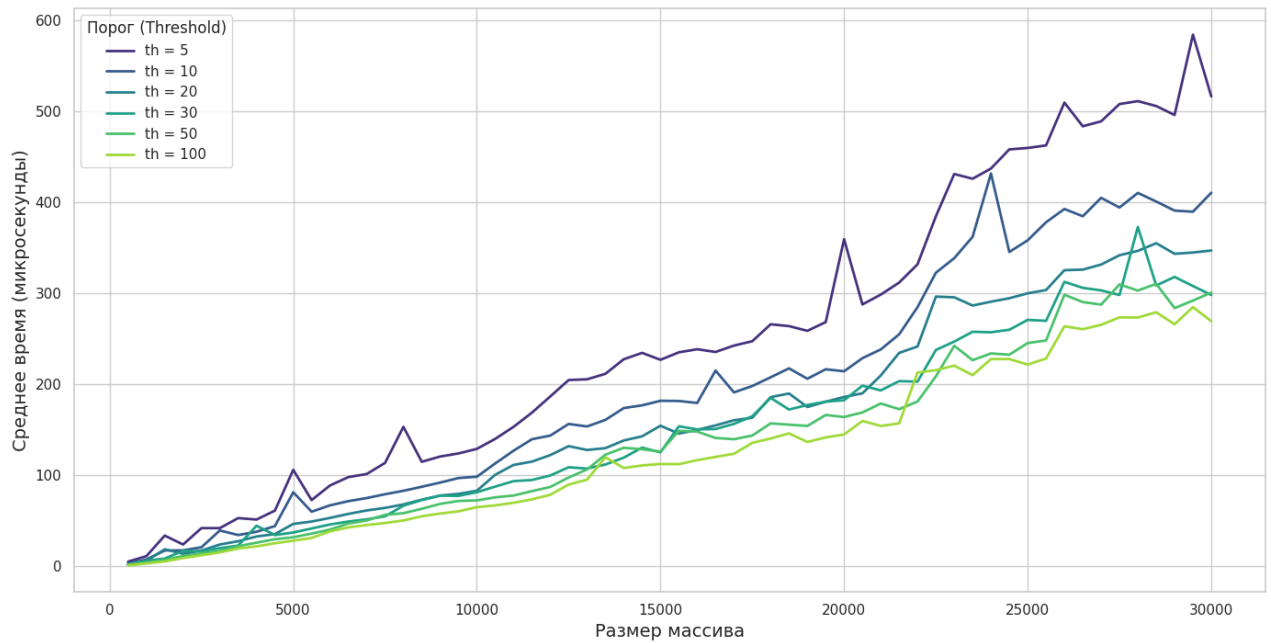


Этап 2: Стандартный Merge Sort
Тип данных: "AlmostSorted"

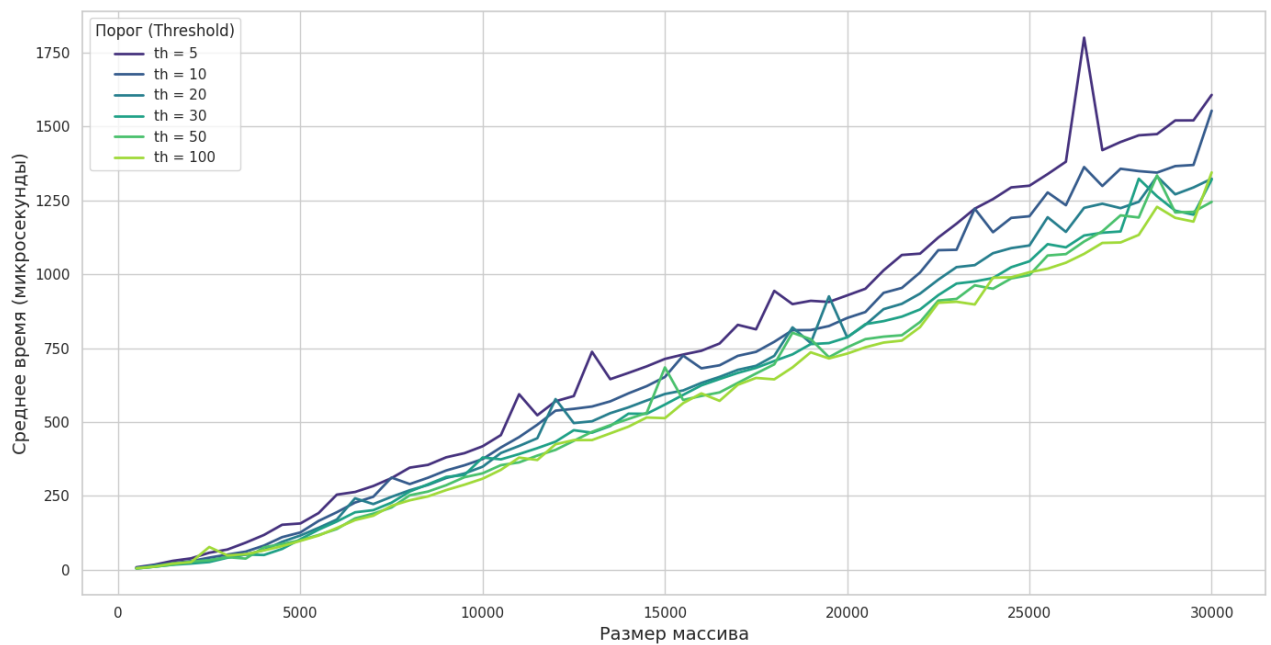


3.2 Графики MERGE+INSERTION SORT

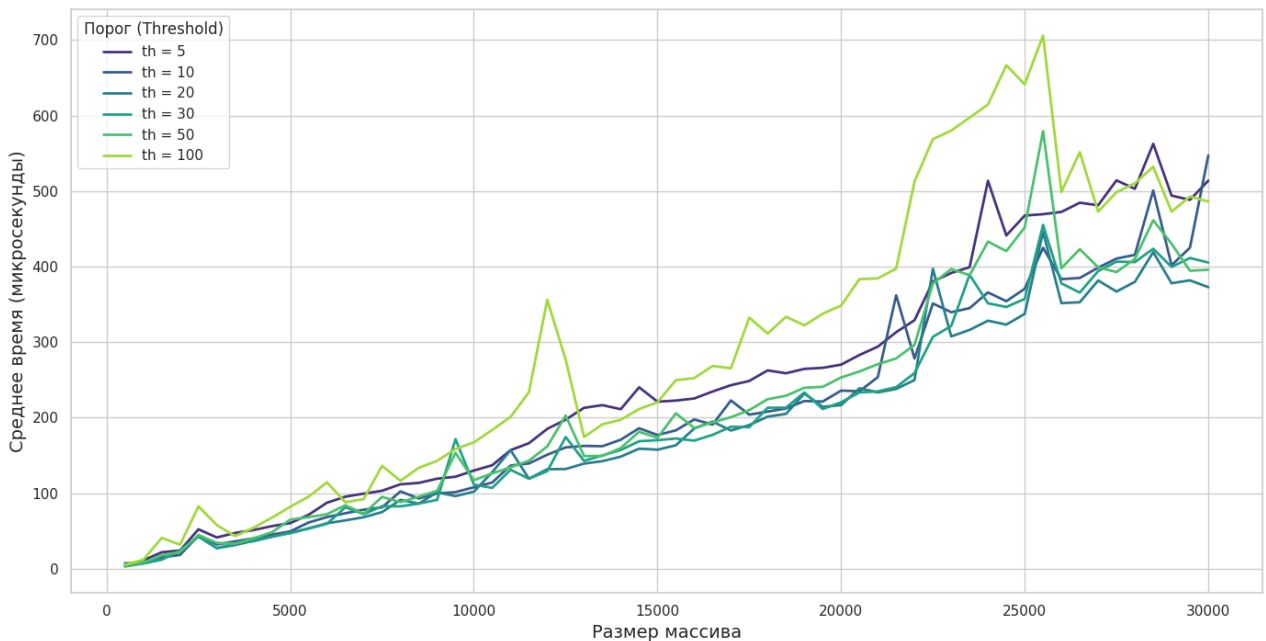
Этап 3: Гибридный Merge+Insertion Sort
Тип данных: "AlmostSorted"



Этап 3: Гибридный Merge+Insertion Sort
Тип данных: "Random"



Этап 3: Гибридный Merge+Insertion Sort
Тип данных: "Reversed"



4 Сравнительный анализ и выводы

Сопоставление результатов для стандартной и гибридной сортировок слиянием позволяет сделать следующие выводы.

На всех типах тестовых данных (случайных, обратно отсортированных и почти отсортированных) гибридная реализация 'MERGE+INSERTION SORT' демонстрирует превосходящую производительность по сравнению со стандартной. Этот выигрыш объясняется устранением высоких накладных расходов на рекурсию для малых подмассивов, где простая итеративная сортировка вставками оказывается быстрее.

Наибольший разрыв в производительности наблюдается на **почти отсортированных данных**, что является лучшим случаем для сортировки вставками (сложность близка к $O(n)$).

Наименьший, но все же заметный выигрыш, гибридный метод показывает на **обратно отсортированных данных**. Этот случай является худшим для сортировки вставками ($O(n^2)$), и именно здесь выбор порога становится критичным. Эксперименты показывают, что слишком большой порог (*threshold*, например, $> 50-100$) может сделать гибридный алгоритм медленнее стандартного, так как квадратичная сложность на большом подмассиве перевешивает экономию на рекурсии.

4.1 Итоговые выводы

- **Эффективность:** Гибридный алгоритм быстрее стандартного при правильном выборе порога.
- **Оптимальный порог:** Оптимальные значения *threshold* находятся в диапазоне 10–50. Это обеспечивает стабильный выигрыш на всех типах данных.
- **Риски:** Слишком большой порог замедляет алгоритм, особенно на обратно отсортированных данных, где можно найти точку, после которой гибридный метод проигрывает стандартному.

Таким образом, гибридизация является оправданной и эффективной практической оптимизацией алгоритма сортировки слиянием.